LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 06 – Inheritance



I Kadek Rio adi Pranata Kusuma

103132400029

S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (Pewarisan) merupakan salah satu pilar utama dalam Pemrograman Berorientasi Objek (OOP) di mana sebuah kelas dapat diturunkan dari kelas lainnya sehingga kelas turunan tersebut dapat mewarisi atribut (*state*) dan method (*behaviour*) dari kelas induknya. Konsep ini membantu menghindari duplikasi kode dan merepresentasikan hubungan hierarkis "is-a" antar objek, di mana kelas induk disebut sebagai *superclass* (base class) dan kelas turunan disebut sebagai *subclass* (derived class). Dalam implementasinya, Java mendukung pewarisan menggunakan kata kunci extends, serta menyediakan keyword super untuk mereferensikan konstruktor atau anggota dari *superclass*-nya.

Selain pewarisan, konsep OOP juga mencakup *Method Overloading* dan *Method Overriding*. *Overloading* terjadi ketika terdapat dua atau lebih method dengan nama yang sama dalam satu kelas namun memiliki parameter yang berbeda (jumlah atau tipe data). Sementara itu, *Overriding* terjadi ketika sebuah *subclass* mendefinisikan ulang method yang sudah ada pada *superclass*-nya, sehingga saat method tersebut dipanggil melalui objek *subclass*, method milik *subclass* lah yang akan dieksekusi.

B. Guided

1. **[Guided 1 - A]**

```
package com.rio.projectmodul06.guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

2. **[Guided 1 - B]**

```
package com.rio.projectmodul06.guided1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = x+y+z;

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

3. [Guided 1 - Demo]

```
package com.rio.projectmodul06.guided1;

public class Demo {
   public static void main(String[] args){
      A superclass = new A();
      B subclass = new B();

      superclass.x = 3;
      superclass.y = 4;

      System.out.println("Superclass: ");
      superclass.tampilXY();

      subclass.x = 1;
      subclass.y = 2;
      subclass.z = 5;

      System.out.println("Subclass: ");
      subclass.jumlahXYZ();
   }
}
```

Output :

```
------------------------< com.Rio:ProjectModul06 >------------------------
Building ProjectModul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul06 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-06

--- compiler:3.15.0:compile (default-compile) @ ProjectModul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.527 s
Finished at: 2026-10-25T20:57:34+07:00
------------------------------------------------------------------------
```

4. [Guided 2 - Employee]

```
package com.rio.projectmodul06.guided2;

public class Employee{
```

```
    private String name;
    String departmen;

    public Employee(String name){
    this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

5. [Guided 2 - Manager]

```
package com.rio.projectmodul06.guided2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departmen);
    }
}
```

6. [Guided 2 - Demo]

```
package com.rio.projectmodul06.guided2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi","Sukabirus");
        adi.departmen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
----------------------< com.Rio:ProjectModul06 >-----------------------
Building ProjectModul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul06 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-06-inheritar

--- compiler:3.15.0:compile (default-compile) @ ProjectModul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
nama: Adi
alamat: Sukabirus
departemen: SDM
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.276 s
Finished at: 2026-10-25T21:08:53+07:00
------------------------------------------------------------------------
```

7. [Guided 3 - Lagu]

```
package com.rio.projectmodul06.guided3;

public class Lagu {
   String pencipta;
   String judul;

   void isiParam(String param1){
      judul = param1;
      pencipta = "Tidak dikenal";
   }

   void isiParam(String param1, String param2){
      judul = param1;
      pencipta = param2;
   }

   void tampilkan(){
      System.out.println("Judul: "+ judul + ", pencipta: " + pencipta);
   }
}
```

8. [Guided 3 - Main]

```
package com.rio.projectmodul06.guided3;

public class Main {
  public static void main(String[] args){
    Lagu d,e;
    d = new Lagu();
```

```
    e = new Lagu();

    d.isiParam("Lagu 1");
    e.isiParam("Lagu 2", "aku");

    d.tampilkan();
    e.tampilkan();
  }
}
```

Output :

```
-----------------------< com.Rio:ProjectModul06 >-----------------------
Building ProjectModul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul06 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-06-inhe

--- compiler:3.15.0:compile (default-compile) @ ProjectModul06 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Judul: Lagu 1, pencipta: Tidak dikenal
Judul: Lagu 2, pencipta: aku
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.380 s
Finished at: 2026-10-25T21:14:34+07:00
------------------------------------------------------------------------
```

9. [Guided 4 - A]

```
public class A {
    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

10. [Guided 4 - B]

```
public class B extends A {
```

```
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik class B dipanggil...");

}
}
```

11. [Guided 4 - Main]

```
public class Main {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :

```
------------------------< com.Rio:ProjectModul06 >------------------------
Building ProjectModul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul06 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-06-inheritance\p

--- compiler:3.15.0:compile (default-compile) @ ProjectModul06 ---
Recompiling the module because of changed source code.
Compiling 12 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.204 s
Finished at: 2026-10-25T21:28:43+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Soal Unguided — Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga

jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.
- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
    - Tanpa parameter: biaya = jumlah data latih × 2.
    - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)

- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.

- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

- Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	`"Regresi Linier"`	500	4 fitur
Klasifikasi	`"Decision Tree"`	800	3 kelas
Klasifikasi *deep learning*	`"Neural Network"`	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
    - biaya 10 *epoch* untuk model `"Neural Network"` (ambil dari array),
    - biaya 10 *epoch* untuk model `"Regresi Linier"`,
    - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model `"Regresi Linier"`.

Contoh Output

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

[Unguded - ModelDeepLearning]

```
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

[Unguided - ModelKlasifikasi]

```
public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }

}
```

[Unguided - ModelML]

```
public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " +
jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

[Unguided - ModelRegresi]

```
public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur)
{
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }
```

```
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

[Unguided - Main]

```
import com.maysa.projectmodul06.Unguided.model.ModelML;
import com.maysa.projectmodul06.Unguided.model.ModelRegresi;
import com.maysa.projectmodul06.Unguided.model.ModelKlasifikasi;
import com.maysa.projectmodul06.Unguided.model.ModelDeepLearning;

public class Main {
    public static void main(String[] args) {

        ModelML[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;
        for (ModelML model : daftarModel) {
            model.tampilkanInfo();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
        System.out.println("Total biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));

        System.out.println("Biaya 10 epoch (Regresi Linier): " +
daftarModel[0].hitungBiaya(10));

        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
-----------------------< com.Rio:ProjectModul06 >-----------------------
Building ProjectModul06 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul06 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-06-inheritance\

--- compiler:3.15.0:compile (default-compile) @ ProjectModul06 ---
Recompiling the module because of changed source code.
Compiling 17 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.387 s
Finished at: 2026-10-25T22:12:03+07:00
```

Penjelasan :

1. **ModelML (Superclass):** Merupakan kelas induk yang merepresentasikan model *machine learning* secara umum. Kelas ini memiliki atribut privat namaModel dan jumlahDataLatih, konstruktor untuk inisialisasi, *getter* getJumlahDataLatih(), method tampilkanInfo() untuk mencetak informasi dasar, serta dua versi method hitungBiaya() yang menerapkan konsep *overloading* (tanpa parameter dan dengan parameter epoch).

2. **ModelRegresi (Subclass pertama dari ModelML):** Mewarisi kelas ModelML dan menambahkan atribut privat jumlahFitur. Kelas ini meng-*override* method tampilkanInfo() untuk menampilkan informasi fitur serta meng-*override* method hitungBiaya() tanpa parameter. Selain itu, terdapat method hitungBiaya(int epoch, int ukuranBatch) yang menambahkan variasi *overloading* untuk menghitung biaya berdasarkan ukuran batch.

3. **ModelKlasifikasi (Subclass kedua dari ModelML):** Mewarisi kelas ModelML dengan tambahan atribut privat jumlahKelas. Kelas ini meng-*override* method tampilkanInfo() dan method hitungBiaya() (mengalikan hasil biaya kelas induk dengan jumlahKelas).

4. **ModelDeepLearning (Subclass turunan dari ModelKlasifikasi):** Mewarisi kelas ModelKlasifikasi (multilevel inheritance) dengan tambahan atribut privat jumlahLayer. Kelas ini meng-*override* method tampilkanInfo() serta method hitungBiaya() untuk memperhitungkan tambahan biaya dari jumlah layer.

5. **Main (Kelas Eksekusi):** Berfungsi untuk menguji seluruh kelas dalam bentuk polimorfisme menggunakan array bertipe ModelML (daftarModel) yang menampung objek ModelRegresi, ModelKlasifikasi, dan ModelDeepLearning. Program melakukan perulangan untuk menampilkan informasi dan biaya masing-masing model, menghitung total keseluruhan biaya, serta memanggil method spesifik seperti perhitungan biaya berbasis *epoch* dan *batch* dengan *downcasting*.

D. Kesimpulan

Berdasarkan praktikum Modul 6 mengenai *Inheritance*, dapat disimpulkan bahwa:

1. Konsep pewarisan (*inheritance*) dan hierarki kelas memungkinkan penggunaan kembali kode (*code reuse*) secara efektif dan terstruktur tanpa harus mendeklarasikan ulang atribut atau method yang sama pada setiap kelas.

2. Implementasi *polimorfisme*, *method overriding*, dan *method overloading* memberikan fleksibilitas tinggi dalam memanggil serta memodifikasi perilaku method sesuai dengan spesifikasi masing-masing kelas turunan (*subclass* maupun *multilevel subclass*).

3. Melalui latihan *unguided*, praktikan mampu menerapkan konsep OOP secara komprehensif dalam studi kasus nyata seperti perhitungan estimasi biaya komputasi berbagai model *machine learning* menggunakan struktur pewarisan yang kompleks, pemanggilan keyword super, serta manipulasi array polimorfik.